ACSE12 Completed Java Programs

Sections 2.1 to 2.5 and 6.1 to 6.5

2.1 Tree Creation and Traversal Techniques (Recursive)

Tree basic terminology

Aim: Build the tree from the PDF and print the parent, children, leaf nodes, and degree of each node.

```
import java.util.*;

class TreeTerminology {
    static void printParents(int node, List<Integer>[] adj, int parent) {
        System.out.println("Parent of " + node + ": " +
                (parent == 0 ? "None" : parent));
        for (int child : adj[node]) {
            if (child != parent) printParents(child, adj, node);
        }
    }

    static void printChildren(int node, List<Integer>[] adj, int parent) {
        System.out.print("Children of " + node + ":");
        boolean found = false;
        for (int child : adj[node]) {
            if (child != parent) {
                System.out.print(" " + child);
                found = true;
            }
        }
        if (!found) System.out.print(" None");
        System.out.println();
        for (int child : adj[node]) {
            if (child != parent) printChildren(child, adj, node);
        }
    }

    static void printLeaves(int node, List<Integer>[] adj, int parent) {
        boolean isLeaf = true;
        for (int child : adj[node]) {
            if (child != parent) {
                isLeaf = false;
                printLeaves(child, adj, node);
            }
        }
        if (isLeaf) System.out.print(node + " ");
    }

    static void printDegrees(int node, List<Integer>[] adj, int parent) {
        int children = 0;
        for (int child : adj[node]) {
            if (child != parent) children++;
        }
        System.out.println("Degree of " + node + ": " + children);
        for (int child : adj[node]) {
            if (child != parent) printDegrees(child, adj, node);
        }
    }

    static void addEdge(List<Integer>[] adj, int u, int v) {
        adj[u].add(v);
        adj[v].add(u);
    }

    public static void main(String[] args) {
        int n = 7, root = 1;
        List<Integer>[] adj = new ArrayList[n + 1];
        for (int i = 0; i <= n; i++) adj[i] = new ArrayList<>();
        addEdge(adj, 1, 2); addEdge(adj, 1, 3); addEdge(adj, 1, 4);
        addEdge(adj, 2, 5); addEdge(adj, 2, 6); addEdge(adj, 4, 7);

        System.out.println("The parents of each node are:");
        printParents(root, adj, 0);
        System.out.println("The children of each node are:");
        printChildren(root, adj, 0);
        System.out.println("The leaf nodes of the tree are:");
        printLeaves(root, adj, 0);
        System.out.println();
        System.out.println("The degrees of each node are:");
        printDegrees(root, adj, 0);
    }
}
```

Output

```
The parents of each node are:
Parent of 1: None
Parent of 2: 1
Parent of 5: 2
Parent of 6: 2
Parent of 3: 1
Parent of 4: 1
Parent of 7: 4
The children of each node are:
Children of 1: 2 3 4
Children of 2: 5 6
Children of 5: None
Children of 6: None
Children of 3: None
Children of 4: 7
Children of 7: None
The leaf nodes of the tree are:
5 6 3 7
The degrees of each node are:
Degree of 1: 3
Degree of 2: 2
Degree of 5: 0
Degree of 6: 0
Degree of 3: 0
Degree of 4: 1
Degree of 7: 0
```

2.1 Tree Creation and Traversal Techniques (Recursive)

Recursive binary tree traversals

Aim: Create the binary tree shown in the PDF and print its inorder, preorder, and postorder traversals.

```
class TreeNode {
    int key;
    TreeNode left, right;
    TreeNode(int key) { this.key = key; }
}

class RecursiveTraversals {
    TreeNode root;

    void printInorder(TreeNode node) {
        if (node == null) return;
        printInorder(node.left);
        System.out.print(node.key + " ");
        printInorder(node.right);
    }

    void printPreorder(TreeNode node) {
        if (node == null) return;
        System.out.print(node.key + " ");
        printPreorder(node.left);
        printPreorder(node.right);
    }

    void printPostorder(TreeNode node) {
        if (node == null) return;
        printPostorder(node.left);
        printPostorder(node.right);
        System.out.print(node.key + " ");
    }

    public static void main(String[] args) {
        RecursiveTraversals tree = new RecursiveTraversals();
        tree.root = new TreeNode(1);
        tree.root.left = new TreeNode(2);
        tree.root.right = new TreeNode(3);
        tree.root.left.left = new TreeNode(4);
        tree.root.left.right = new TreeNode(5);

        System.out.println("Inorder traversal of binary tree is");
        tree.printInorder(tree.root);
        System.out.println();
        System.out.println("Preorder traversal of binary tree is");
        tree.printPreorder(tree.root);
        System.out.println();
        System.out.println("Postorder traversal of binary tree is");
        tree.printPostorder(tree.root);
        System.out.println();
    }
}
```

Output

```
Inorder traversal of binary tree is
4 2 5 1 3
Preorder traversal of binary tree is
1 2 4 5 3
Postorder traversal of binary tree is
4 5 2 3 1
```

2.2 Tree Creation and Traversal Techniques (Iterative)

Iterative inorder traversal

Aim: Print the inorder traversal of the PDF sample tree using an explicit stack.

```
import java.util.*;

class TreeNode {
    int data;
    TreeNode left, right;
    TreeNode(int data) { this.data = data; }
}

class IterativeInorder {
    TreeNode root;

    void inorder() {
        Deque<TreeNode> stack = new ArrayDeque<>();
        TreeNode current = root;
        while (current != null || !stack.isEmpty()) {
            while (current != null) {
                stack.push(current);
                current = current.left;
            }
            current = stack.pop();
            System.out.print(current.data + " ");
            current = current.right;
        }
    }

    public static void main(String[] args) {
        IterativeInorder tree = new IterativeInorder();
        tree.root = new TreeNode(1);
        tree.root.left = new TreeNode(2);
        tree.root.right = new TreeNode(3);
        tree.root.left.left = new TreeNode(4);
        tree.root.left.right = new TreeNode(5);
        tree.inorder();
        System.out.println();
    }
}
```

Output

```
4 2 5 1 3
```

2.2 Tree Creation and Traversal Techniques (Iterative)

Iterative preorder traversal

Aim: Print the preorder traversal of the PDF sample tree using a stack.

```
import java.util.*;

class TreeNode {
    int data;
    TreeNode left, right;
    TreeNode(int data) { this.data = data; }
}

class IterativePreorder {
    TreeNode root;

    void iterativePreorder(TreeNode node) {
        if (node == null) return;
        Deque<TreeNode> stack = new ArrayDeque<>();
        stack.push(node);
        while (!stack.isEmpty()) {
            TreeNode current = stack.pop();
            System.out.print(current.data + " ");
            if (current.right != null) stack.push(current.right);
            if (current.left != null) stack.push(current.left);
        }
    }

    public static void main(String[] args) {
        IterativePreorder tree = new IterativePreorder();
        tree.root = new TreeNode(10);
        tree.root.left = new TreeNode(8);
        tree.root.right = new TreeNode(2);
        tree.root.left.left = new TreeNode(3);
        tree.root.left.right = new TreeNode(5);
        tree.root.right.left = new TreeNode(2);
        tree.iterativePreorder(tree.root);
        System.out.println();
    }
}
```

Output

```
10 8 3 5 2 2
```

2.2 Tree Creation and Traversal Techniques (Iterative)

Iterative postorder traversal

Aim: Print the postorder traversal of the PDF sample tree using one stack and a last-visited pointer.

```
import java.util.*;

class TreeNode {
    int data;
    TreeNode left, right;
    TreeNode(int data) { this.data = data; }
}

class IterativePostorder {
    TreeNode root;

    List<Integer> postOrderIterative(TreeNode node) {
        List<Integer> result = new ArrayList<>();
        Deque<TreeNode> stack = new ArrayDeque<>();
        TreeNode current = node, lastVisited = null;
        while (current != null || !stack.isEmpty()) {
            if (current != null) {
                stack.push(current);
                current = current.left;
            } else {
                TreeNode peek = stack.peek();
                if (peek.right != null && lastVisited != peek.right) {
                    current = peek.right;
                } else {
                    result.add(peek.data);
                    lastVisited = stack.pop();
                }
            }
        }
        return result;
    }

    public static void main(String[] args) {
        IterativePostorder tree = new IterativePostorder();
        tree.root = new TreeNode(1);
        tree.root.left = new TreeNode(2);
        tree.root.right = new TreeNode(3);
        tree.root.left.left = new TreeNode(4);
        tree.root.left.right = new TreeNode(5);
        tree.root.right.left = new TreeNode(6);
        tree.root.right.right = new TreeNode(7);
        System.out.println("Post order traversal of binary tree is:");
        System.out.println(tree.postOrderIterative(tree.root));
    }
}
```

Output

```
Post order traversal of binary tree is:
[4, 5, 2, 6, 7, 3, 1]
```

2.3 Hashing

Count quadruples

Aim: Count quadruples, with one value from each of four arrays, whose sum equals x.

```
import java.util.*;

class QuadrupleCounter {
    static long countQuadruples(int[] a, int[] b, int[] c, int[] d, long x) {
        Map<Long, Long> pairFrequency = new HashMap<>();
        for (int first : a) {
            for (int second : b) {
                long sum = (long) first + second;
                pairFrequency.put(sum, pairFrequency.getOrDefault(sum, 0L) + 1);
            }
        }
        long count = 0;
        for (int third : c) {
            for (int fourth : d) {
                long needed = x - third - fourth;
                count += pairFrequency.getOrDefault(needed, 0L);
            }
        }
        return count;
    }

    public static void main(String[] args) {
        int[] arr1 = {1, 4, 5, 6};
        int[] arr2 = {2, 3, 7, 8};
        int[] arr3 = {1, 4, 6, 10};
        int[] arr4 = {2, 4, 7, 8};
        int x = 30;
        System.out.println("Count = " +
                countQuadruples(arr1, arr2, arr3, arr4, x));
    }
}
```

Output

```
Count = 4
```

2.4 Graph Traversal Techniques

Breadth-first traversal (BFS)

Aim: Print the breadth-first traversal from source vertex 0 using an adjacency list and a queue.

```
import java.util.*;

class GraphBFS {
    int vertices;
    LinkedList<Integer>[] adjList;

    @SuppressWarnings("unchecked")
    GraphBFS(int vertices) {
        this.vertices = vertices;
        adjList = new LinkedList[vertices];
        for (int i = 0; i < vertices; i++) adjList[i] = new LinkedList<>();
    }

    void addEdge(int u, int v) { adjList[u].add(v); }

    void bfs(int startNode) {
        boolean[] visited = new boolean[vertices];
        Queue<Integer> queue = new ArrayDeque<>();
        visited[startNode] = true;
        queue.add(startNode);
        while (!queue.isEmpty()) {
            int node = queue.remove();
            System.out.print(node + " ");
            for (int neighbor : adjList[node]) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    queue.add(neighbor);
                }
            }
        }
    }

    public static void main(String[] args) {
        GraphBFS graph = new GraphBFS(5);
        graph.addEdge(0, 1); graph.addEdge(0, 2);
        graph.addEdge(1, 3); graph.addEdge(1, 4);
        graph.addEdge(2, 4);
        System.out.print("Breadth First Traversal starting from vertex 0: ");
        graph.bfs(0);
        System.out.println();
    }
}
```

Output

```
Breadth First Traversal starting from vertex 0: 0 1 2 3 4
```

2.4 Graph Traversal Techniques

Depth-first traversal (DFS)

Aim: Print a depth-first traversal from source vertex 2 using recursion and a visited array.

```
import java.util.*;

class GraphDFS {
    int vertices;
    LinkedList<Integer>[] adj;

    @SuppressWarnings("unchecked")
    GraphDFS(int vertices) {
        this.vertices = vertices;
        adj = new LinkedList[vertices];
        for (int i = 0; i < vertices; i++) adj[i] = new LinkedList<>();
    }

    void addEdge(int from, int to) { adj[from].add(to); }

    void dfsUtil(int vertex, boolean[] visited) {
        visited[vertex] = true;
        System.out.print(vertex + " ");
        for (int next : adj[vertex]) {
            if (!visited[next]) dfsUtil(next, visited);
        }
    }

    void dfs(int start) {
        boolean[] visited = new boolean[vertices];
        dfsUtil(start, visited);
    }

    public static void main(String[] args) {
        GraphDFS graph = new GraphDFS(4);
        graph.addEdge(0, 1); graph.addEdge(0, 2);
        graph.addEdge(1, 2); graph.addEdge(2, 0);
        graph.addEdge(2, 3); graph.addEdge(3, 3);
        System.out.println("Following is Depth First Traversal " +
                "(starting from vertex 2)");
        graph.dfs(2);
        System.out.println();
    }
}
```

Output

```
Following is Depth First Traversal (starting from vertex 2)
2 0 1 3
```

2.5 Heap Sort

Heap sort

Aim: Sort the PDF sample array in ascending order using an in-place max-heap.

```
class HeapSort {
    void sort(int[] arr) {
        int n = arr.length;
        for (int i = n / 2 - 1; i >= 0; i--) heapify(arr, n, i);
        for (int end = n - 1; end > 0; end--) {
            int temp = arr[0];
            arr[0] = arr[end];
            arr[end] = temp;
            heapify(arr, end, 0);
        }
    }

    void heapify(int[] arr, int size, int root) {
        int largest = root;
        int left = 2 * root + 1;
        int right = 2 * root + 2;
        if (left < size && arr[left] > arr[largest]) largest = left;
        if (right < size && arr[right] > arr[largest]) largest = right;
        if (largest != root) {
            int temp = arr[root];
            arr[root] = arr[largest];
            arr[largest] = temp;
            heapify(arr, size, largest);
        }
    }

    static void printArray(int[] arr) {
        for (int value : arr) System.out.print(value + " ");
        System.out.println();
    }

    public static void main(String[] args) {
        int[] arr = {12, 11, 13, 5, 6, 7};
        HeapSort sorter = new HeapSort();
        sorter.sort(arr);
        System.out.println("Sorted array is");
        printArray(arr);
    }
}
```

Output

```
Sorted array is
5 6 7 11 12 13
```

6.1 Assign Mice to Holes

Assign mice to holes

Aim: Assign each mouse to a unique hole so the maximum time for a mouse to reach its hole is minimized.

```
import java.util.*;

class AssignMiceToHoles {
    static int assignHole(ArrayList<Integer> mice, ArrayList<Integer> holes) {
        if (mice.size() != holes.size()) return -1;
        Collections.sort(mice);
        Collections.sort(holes);
        int maxTime = 0;
        for (int i = 0; i < mice.size(); i++) {
            maxTime = Math.max(maxTime, Math.abs(mice.get(i) - holes.get(i)));
        }
        return maxTime;
    }

    public static void main(String[] args) {
        ArrayList<Integer> mice = new ArrayList<>(Arrays.asList(4, -4, 2));
        ArrayList<Integer> holes = new ArrayList<>(Arrays.asList(4, 0, 5));
        System.out.println("The last mouse gets into the hole in time: " +
                assignHole(mice, holes));
    }
}
```

Output

```
The last mouse gets into the hole in time: 4
```

6.2 Minimum Cost to Cut a Board into Squares

Minimum board-cutting cost

Aim: Cut the board into unit squares at minimum total cost by choosing the highest-cost available cut first.

```
import java.util.*;

class BoardCutting {
    static long minimumCostOfBreaking(Integer[] horizontal, Integer[] vertical,
                                      int horizontalCount, int verticalCount) {
        Arrays.sort(horizontal, Collections.reverseOrder());
        Arrays.sort(vertical, Collections.reverseOrder());
        int i = 0, j = 0;
        long total = 0;
        long horizontalPieces = 1, verticalPieces = 1;

        while (i < horizontalCount && j < verticalCount) {
            if (horizontal[i] >= vertical[j]) {
                total += horizontal[i] * verticalPieces;
                horizontalPieces++;
                i++;
            } else {
                total += vertical[j] * horizontalPieces;
                verticalPieces++;
                j++;
            }
        }
        while (i < horizontalCount) {
            total += horizontal[i++] * verticalPieces;
        }
        while (j < verticalCount) {
            total += vertical[j++] * horizontalPieces;
        }
        return total;
    }

    public static void main(String[] args) {
        int m = 6, n = 4;
        Integer[] X = {2, 1, 3, 1, 4};
        Integer[] Y = {4, 1, 2};
        System.out.println(minimumCostOfBreaking(X, Y, m - 1, n - 1));
    }
}
```

Output

```
42
```

6.3 Connect N Ropes with Minimum Cost

Connect ropes

Aim: Connect all ropes into one rope while minimizing the sum of connection costs.

```
import java.util.*;

class ConnectRopes {
    static long minCost(int[] lengths) {
        PriorityQueue<Long> minHeap = new PriorityQueue<>();
        for (int length : lengths) minHeap.add((long) length);
        long totalCost = 0;
        while (minHeap.size() > 1) {
            long first = minHeap.remove();
            long second = minHeap.remove();
            long combined = first + second;
            totalCost += combined;
            minHeap.add(combined);
        }
        return totalCost;
    }

    public static void main(String[] args) {
        int[] lengths = {4, 3, 2, 6};
        System.out.println("Total cost for connecting ropes is " +
                minCost(lengths));
    }
}
```

Output

```
Total cost for connecting ropes is 29
```

6.4 Rearrange a String so Same Characters Become d Distance Away

Rearrange string characters

Aim: Rearrange a string so each pair of equal characters is at least d positions apart, or report that it is impossible.

```
import java.util.*;

class RearrangeString {
    static class CharCount {
        char ch;
        int count;
        CharCount(char ch, int count) { this.ch = ch; this.count = count; }
    }

    static class Cooling {
        CharCount item;
        int releaseAt;
        Cooling(CharCount item, int releaseAt) {
            this.item = item;
            this.releaseAt = releaseAt;
        }
    }

    static boolean rearrange(char[] input, char[] output, int d) {
        if (d <= 1) {
            System.arraycopy(input, 0, output, 0, input.length);
            return true;
        }
        int[] frequency = new int[256];
        for (char ch : input) {
            if (ch >= 256) return false;
            frequency[ch]++;
        }
        PriorityQueue<CharCount> available = new PriorityQueue<>(
                (a, b) -> a.count != b.count
                        ? Integer.compare(b.count, a.count)
                        : Character.compare(a.ch, b.ch));
        for (int ch = 0; ch < frequency.length; ch++) {
            if (frequency[ch] > 0) available.add(new CharCount((char) ch, frequency[ch]));
        }
        Queue<Cooling> cooling = new ArrayDeque<>();
        for (int position = 0; position < input.length; position++) {
            while (!cooling.isEmpty() && cooling.peek().releaseAt <= position) {
                available.add(cooling.remove().item);
            }
            if (available.isEmpty()) return false;
            CharCount item = available.remove();
            output[position] = item.ch;
            item.count--;
            if (item.count > 0) cooling.add(new Cooling(item, position + d));
        }
        return true;
    }

    public static void main(String[] args) {
        char[] input = "aaaabbbcc".toCharArray();
        char[] output = new char[input.length];
        if (rearrange(input, output, 2))
            System.out.println(String.valueOf(output));
        else
            System.out.println("Cannot be rearranged");
    }
}
```

Output

```
ababacabc
```

6.5 Minimization of Cash Flow among Friends

Minimize cash flow

Aim: Settle the debts among the people by repeatedly transferring money between the largest debtor and largest creditor.

```
class CashFlow {
    static final int N = 3;

    static int getMin(int[] amount) {
        int minIndex = 0;
        for (int i = 1; i < N; i++) {
            if (amount[i] < amount[minIndex]) minIndex = i;
        }
        return minIndex;
    }

    static int getMax(int[] amount) {
        int maxIndex = 0;
        for (int i = 1; i < N; i++) {
            if (amount[i] > amount[maxIndex]) maxIndex = i;
        }
        return maxIndex;
    }

    static void minCashFlowRec(int[] amount) {
        int debtor = getMin(amount);
        int creditor = getMax(amount);
        if (amount[debtor] == 0 && amount[creditor] == 0) return;

        int transfer = Math.min(-amount[debtor], amount[creditor]);
        amount[debtor] += transfer;
        amount[creditor] -= transfer;
        System.out.println("Person " + debtor + " pays " + transfer +
                " to Person " + creditor);
        minCashFlowRec(amount);
    }

    static void minCashFlow(int[][] graph) {
        int[] amount = new int[N];
        for (int from = 0; from < N; from++) {
            for (int to = 0; to < N; to++) {
                amount[from] -= graph[from][to];
                amount[to] += graph[from][to];
            }
        }
        minCashFlowRec(amount);
    }

    public static void main(String[] args) {
        int[][] graph = {
            {0, 1000, 2000},
            {0, 0, 5000},
            {0, 0, 0}
        };
        minCashFlow(graph);
    }
}
```

Output

```
Person 1 pays 4000 to Person 2
Person 0 pays 3000 to Person 2
```